СЕМИНАР ПО ЛЕКЦИИ 5 · ДОМАШНЕЕ ЗАДАНИЕ

Домашнее задание 5. Витрины интернет-магазина по схеме «звезда»

Это продолжение семинара и **первый слой сквозного проекта курса**: аналитический слой в хранилище Greenplum. Дальше его сборку будет запускать Airflow, а рядом появятся потоковые данные из Kafka и ClickHouse. Поэтому делайте проект так, как будто им будут пользоваться другие: понятные имена, тесты и описания.

ПОСТАНОВКА

Задача

Вам дана сырая OLTP-база интернет-магазина (PostgreSQL). Сервис `loader` копирует её таблицы как есть в Greenplum, в базу `dwh`, схему `raw`. Это источник для dbt, он описан в `models/staging/shop/_shop__sources.yml`:

Таблица	Одна строка — это	Ключ
`customers`	клиент	`customer_id`
`products`	товар; категория записана текстом	`product_id`
`product_categories`	категория; дерево до 3 уровней через `parent_category_id`	`category_id`
`orders`	заказ	`order_id`
`order_items`	товар в заказе	`(order_id, product_id)`
`payments`	платёжная операция: попытка, оплата частью, возврат	`payment_id`

Нужно:

1. **Спроектировать** аналитические витрины по схеме «звезда».
2. **Реализовать** их в dbt со слоями staging / intermediate / marts в Greenplum и выбрать, как хранить каждую витрину.
3. **Написать тесты** на ключевые поля и на бизнес-логику.
4. **Собрать документацию** проекта.

Бизнес хочет отвечать на вопросы такого вида: выручка по месяцам и категориям товаров, доля покупателей по каналам привлечения, средний чек по городам, какие бренды чаще возвращают.

Обязательные витрины

Это мы в intermidiate делаем product_categories

Витрина	Тип	Зернистость (одна строка — это…)	Что должно быть
`fct_orders`	факт	заказ	Есть после семинара. Добавьте `items_count` — число разных товаров в заказе. Тестовых заказов нет
`fct_order_items`	факт	товар в заказе	Ключи на все измерения: `product_id`, `customer_id`, `order_date`; меры `quantity`, `unit_price`, `discount_amount`, `line_amount`; признак `is_confirmed` заказа
`dim_customers`	измерение	клиент, **включая не купивших**	Атрибуты клиента; первая подтверждённая покупка (`first_confirmed_order_id`, `first_confirmed_at`); `confirmed_orders_count`, `confirmed_revenue`, `is_buyer`
`dim_products`	измерение	товар	Название, бренд и иерархия: `category_name`, `parent_category_name`, `root_category_name` — верхний уровень (`Electronics` или `Home`)
`dim_dates`	измерение	день	Календарь, покрывающий все даты заказов: год, квартал, месяц, день недели, `is_weekend`

Названия колонок, кроме перечисленных, выбирайте сами. Можно добавить свои витрины, если сможете объяснить, какой бизнес-вопрос они закрывают.

Готово, все в папке staging, просто перекинули из

ТРЕБОВАНИЯ

Требования к слоям

Слой	Схема в базе	Материализация	Правила
staging	`analytics_staging`	view	По одной модели на **каждую** таблицу источника, имена `stg_shop__<таблица>`. Без JOIN и агрегаций. Только здесь используется `source()`
intermediate	`analytics_intermediate`	view или ephemeral	JOIN, агрегации, сложная логика. Нужны минимум `int_orders_with_payments` и модель, которая разворачивает дерево категорий
marts	`analytics_marts`	table	Только `fct_` и `dim_`. Читают только `ref()`

Подсказка про дерево категорий: глубина сейчас равна 3 (`Home > Kitchen > Cookware`). Решение с рекурсивным CTE (`with recursive`) не сломается, если появится четвёртый уровень.

Требования к хранению в Greenplum

Для **каждой** витрины marts выберите способ хранения и задайте его в `config()` модели. Макрос `macros/greenplum_create_table_as.sql` понимает три настройки:

Тут я напихал по разному. Маленькие таблички distributed_replicated=true большие {{ config(materialized='table', distributed_by='order_id', orientation='column') }}

Настройка	Что получится в Greenplum	Когда подходит
`distributed_by='столбец'`	`distributed by (столбец)`	Большие таблицы: ключ, по которому таблицу чаще всего соединяют, с равномерными значениями
`distributed_replicated=true`	`distributed replicated` — копия на каждом сегменте	Маленькие измерения: JOIN с ними идёт без пересылки строк
`orientation='column'`	Колоночная append-optimized таблица	Широкие факты, из которых отчёты читают несколько столбцов

Без настроек витрина будет `distributed randomly`. Это допустимо, но такой выбор тоже нужно обосновать.

В `README.md` добавьте таблицу «Хранение в Greenplum»: для каждой витрины — выбранное распределение и одна-две фразы, почему. Подумайте, как связаны ключи `fct_orders`, `fct_order_items` и сырых таблиц `raw` (см. `init/greenplum/load_raw.sql`) и что будет с соединением `fct_order_items` и `fct_orders`, если ключи у них разные.

Проверить, что получилось:

```
select c.relname, pg_get_table_distributedby(c.oid) as distribution, am.amname as storage
from pg_class c
join pg_namespace n on n.oid = c.relnamespace
left join pg_am am on am.oid = c.relam
where n.nspname = 'analytics_marts' and c.relkind = 'r';
```

Требования к тестам

Generic-тесты:

- `unique` и `not_null` на первичный ключ **каждой** модели staging и marts;
- `relationships` от каждого внешнего ключа фактов к своему измерению: `fct_order_items.product_id → dim_products`, `… customer_id → dim_customers`, `… order_date → dim_dates`, `fct_orders.customer_id → dim_customers`;
- `accepted_values` на статусы заказа и платежа;
- **свой generic-тест** в `tests/generic/` (например, `non_negative`: значение не меньше нуля), применённый минимум к двум колонкам.

Singular-тесты на бизнес-логику, минимум 4:

1. Сумма заказа в `fct_orders` равна сумме `line_amount` его позиций в `fct_order_items`.
2. Сверка с источником: сумма подтверждённых покупок в витрине совпадает с суммой, посчитанной прямо по `source('shop', 'orders')` и `source('shop', 'payments')`.
3. По возвращённому заказу деньги вернулись полностью: `net_paid_amount = 0`.
4. По подтверждённому заказу не списали больше его суммы. Этот тест **найдёт** реальную проблему. Не подгоняйте под него данные: поставьте `severity: warn` и опишите в документации, что найдено.

Из dbt-project запускается

Можно добавить свои проверки, например: первая покупка не раньше регистрации.

Требования к документации

- `description` у каждой модели marts и у **каждой** их колонки; у staging — у моделей и ключевых колонок.
- Определение подтверждённой покупки — в **doc-блоке** (`{% docs confirmed_order %}` в файле `models/docs.md`). Используйте его в описаниях `is_confirmed` через `{{ doc("confirmed_order") }}`.
- Страница обзора проекта: doc-блок `__overview__`.
- В `README.md` вашего репозитория добавьте раздел «Модель данных»:
  - ER-диаграмму звезды: Mermaid `erDiagram` (GitHub рисует его сам) или картинку;
  - для каждой витрины одну строку: зернистость и на какой бизнес-вопрос она отвечает;
  - таблицу «Хранение в Greenplum» (см. выше);
  - скриншот графа lineage из `dbt docs serve` (файл `docs/lineage.png`).

Каркас для Mermaid:

```
```mermaid
erDiagram
    dim_customers ||--o{ fct_orders : "customer_id"
    fct_orders ||--|{ fct_order_items : "order_id"
```
```

Дополнительное задание (обязательно)

Выполните одно на выбор:

- сделать `fct_order_items` инкрементальной моделью (`materialized='incremental'`, `unique_key`, `is_incremental()`). В README объясните, какие изменения в источнике такая модель **пропустит**;
- подключить пакет `dbt_utils` (`packages.yml`, `dbt deps`) и использовать его для календаря (`date_spine`) или ключа (`generate_surrogate_key`).

СДАЧА

Как сдавать

1. Создайте **свой** пустой репозиторий на GitHub или GitLab, приватный или публичный, и дайте доступ преподавателю.
2. Привяжите к нему локальный проект, склонированный на семинаре, и отправьте код:

```
git remote rename origin course              # исходный репозиторий курса
git remote add origin <ссылка на ваш репозиторий>
git push -u origin main
```

3. Пришлите ссылку на репозиторий.

Перед отправкой проверьте с нуля так же, как будет проверять преподаватель:

```
docker compose down -v && docker compose up -d   # чистые базы и повторная загрузка raw
docker compose logs loader                       # дождитесь строки «Готово»
```

```
docker compose run --rm dbt build                    # ожидается ERROR=0; WARN допустим, если объяснён
```

Папки `target/`, `logs/`, `dbt_packages/` в репозиторий не кладите: они уже в `.gitignore`.

Самопроверка

Проверка	Ожидаемый результат
Где лежат витрины	Greenplum, база `dwh`, схема `analytics_marts`
`fct_orders`	`135` строк, тестовых заказов нет
Подтверждённые заказы / сумма	`106` / `91 620,00`
`fct_order_items`	`425` строк (из `444` позиций исключены позиции тестовых заказов)
Сумма `line_amount` по подтверждённым заказам	`91 620,00` — совпадает с `fct_orders`
`dim_customers`	`40` строк: `29` покупателей, `11` без покупок
Клиент `1`	Первая подтверждённая покупка — заказ `101`, всего `9` покупок на `7 280,00`
Клиент `40`	`is_buyer = false`, первая покупка — `null`
`dim_products`	`13` строк. Товар `11` (`Pot A`): `Home > Kitchen > Cookware`; товар `12` (`Case B`): `Electronics > Accessories > Cases`
Выручка подтверждённых заказов по `root_category_name`	`Electronics` — `59 994,28`, `Home` — `31 625,72`
`dim_dates`	`133` дня: с `2026-03-01` по `2026-07-11`
Тест «не списали больше суммы заказа»	`WARN`: заказ `113`, двойное списание `2 × 200`

СПРАВОЧНИК

Краткий синтаксис dbt

Шпаргалка по всему, что понадобится в задании. Команды запускайте из папки проекта через `docker compose run --rm dbt …` (или через алиас `dbt`).

Ссылки на таблицы

Запись	Что делает	Где использовать
`{{ source('shop', 'orders') }}`	Сырая таблица `raw.orders`, описанная в `_shop__sources.yml`	Только в staging
`{{ ref('stg_shop__orders') }}`	Другая модель проекта; dbt сам подставит схему и выстроит порядок запуска	Везде, кроме staging
`{{ this }}`	Таблица текущей модели	В incremental-моделях

Настройки модели

В начале `.sql`-файла модели:

```
{{ config(
    materialized='table',
    distributed_by='order_id',
    orientation='column'
) }}
```

Настройка	Значения
`materialized`	`view`, `table`, `incremental`, `ephemeral`
`distributed_by` / `distributed_replicated`	Распределение в Greenplum: `'столбец'` или `true`
`orientation`	`'column'` — колоночное хранение в Greenplum

Внутри `{{ … }}` нельзя писать SQL-комментарии `--`: dbt не скомпилирует модель.

Настройка для целой папки — в `dbt_project.yml` с плюсом: `+materialized: view`, `+schema: staging`.

Jinja

Запись	Смысл
`{{ … }}`	Вывести значение в SQL
`{% … %}`	Команда: `set`, `if`, `for`, `macro`
`{# … #}`	Комментарий, в SQL не попадает

```
{% set statuses = ['success', 'failed', 'pending'] %}
select
    order_id,
    {% for s in statuses %}
    count(*) filter (where payment_status = '{{ s }}') as {{ s }}_count{% if not loop.last %},{%
endif %}
    {% endfor %}
from {{ ref('stg_shop__payments') }}
group by order_id
```

Посмотреть, во что превратилась модель: `dbt compile -s <модель>`, затем файл в `target/compiled/…`.

Макрос

Файл в `macros/`, вызывается в любой модели как функция:

```
{% macro rub(column) %}
    round({{ column }}::numeric, 2)
{% endmacro %}

-- в модели:
select {{ rub('amount') }} as amount_rub from …
```

Описания и generic-тесты (YAML)

```
version: 2
models:
    - name: fct_orders
    description: "Заказы без тестовых. Одна строка — один заказ."
    columns:
      - name: order_id
        description: "Номер заказа."
        data_tests: [unique, not_null]
      - name: customer_id
        data_tests:
          - relationships:
              arguments:
                to: ref('dim_customers')
                field: customer_id
      - name: order_status
        data_tests:
          - accepted_values:
              arguments:
                values: ['paid', 'pending', 'returned', 'cancelled']
              config:
                severity: warn       # предупреждение вместо ошибки
      - name: is_confirmed
        description: '{{ doc("confirmed_order") }}'
```

Свой generic-тест

Файл `tests/generic/non_negative.sql`. Тест возвращает «плохие» строки; ноль строк — тест пройден.

```
{% test non_negative(model, column_name) %}
select * from {{ model }} where {{ column_name }} < 0
{% endtest %}
```

В YAML подключается по имени: `data_tests: [non_negative]`.

Singular-тест

Любой `.sql`-файл в `tests/`, который возвращает строки-нарушения:

```
{{ config(severity='warn') }}   -- если нужно предупреждение, а не ошибка

select o.order_id
from {{ ref('fct_orders') }} o
where o.order_amount < 0
```

Doc-блоки

Файл `models/docs.md`:

```
{% docs __overview__ %}
# Аналитический слой интернет-магазина
Текст главной страницы документации.
{% enddocs %}

{% docs confirmed_order %}
Определение подтверждённой покупки.
{% enddocs %}
```

Incremental-модель

```
{{ config(materialized='incremental', unique_key='order_item_id') }}

select …
from {{ ref('stg_shop__orders') }}
{% if is_incremental() %}
where ordered_at > (select max(ordered_at) from {{ this }})
{% endif %}
```

Первый запуск и смена структуры таблицы — с флагом `--full-refresh`.

Пакет dbt_utils

Файл `packages.yml` в корне проекта, затем `dbt deps`:

```
packages:
    - package: dbt-labs/dbt_utils
    version: [">=1.3.0", "<2.0.0"]
```

```
{{ dbt_utils.date_spine(datepart="day",
     start_date="cast('2026-03-01' as date)",
     end_date="cast('2026-07-12' as date)") }}

{{ dbt_utils.generate_surrogate_key(['order_id', 'product_id']) }}
```

Команды

Команда	Что делает
`dbt debug`	Проверить подключение к Greenplum
`dbt run`	Собрать модели
`dbt test`	Запустить тесты
`dbt build`	Модели и тесты вместе, по порядку зависимостей
`dbt compile -s <модель>`	Показать итоговый SQL без запуска
`dbt deps`	Скачать пакеты из `packages.yml`
`dbt docs generate`	Собрать документацию
`dbt run --full-refresh -s <модель>`	Пересоздать incremental-модель с нуля
`dbt docs serve --host 0.0.0.0 --port 8080 --no-browser`	Открыть документацию на http://localhost:8080 (запуск через `docker compose run --rm --service-ports dbt …`)
`-s fct_orders`	Выбрать одну модель
`-s +fct_orders` / `-s fct_orders+`	Модель и всё, от чего она зависит / что зависит от неё
`-s staging`, `-s models/marts`	Папку целиком
`--exclude stg_shop__payments`	Всё, кроме модели